DATA 255 Deep Learning Technologies

Oct 30, 2025

Taehee Jeong, Ph.D.

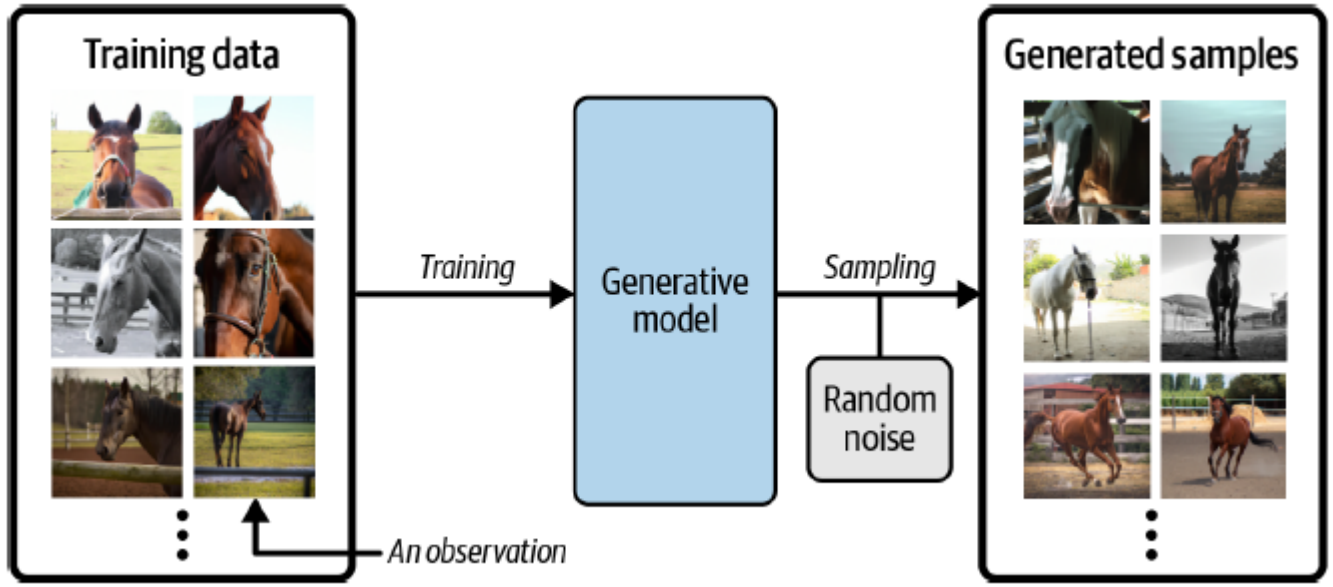
SJSU SAN JOSÉ STATE UNIVERSITY

Topics(1)

- Generative Modeling
  - Autoencoder
  - VAE

Generative Modeling

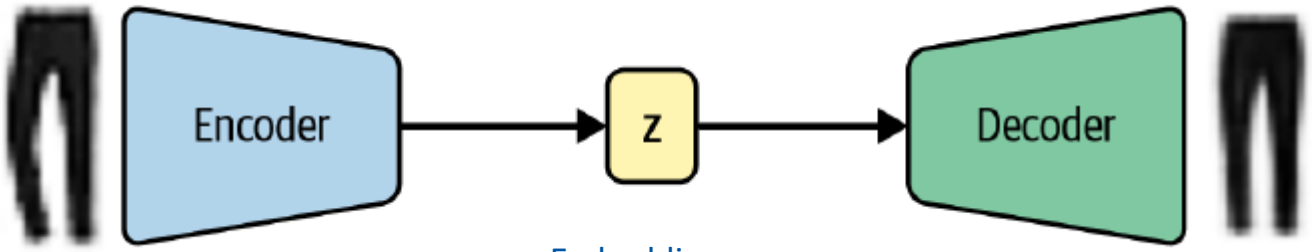
Generative modeling is a branch of machine learning that involves training a model to produce new data that is similar to a given dataset



Source: Generative Deep Learning(2nd Ed), David Foster

Autoencoder Architecture

- An encoder network that compresses high-dimensional input data such as an image into a lower-dimensional embedding vector
- A decoder network that decompresses a given embedding vector back to the original domain (e.g., back to an image)



Embedding space
(latent space)

Source: Generative Deep Learning(2nd Ed), David Foster

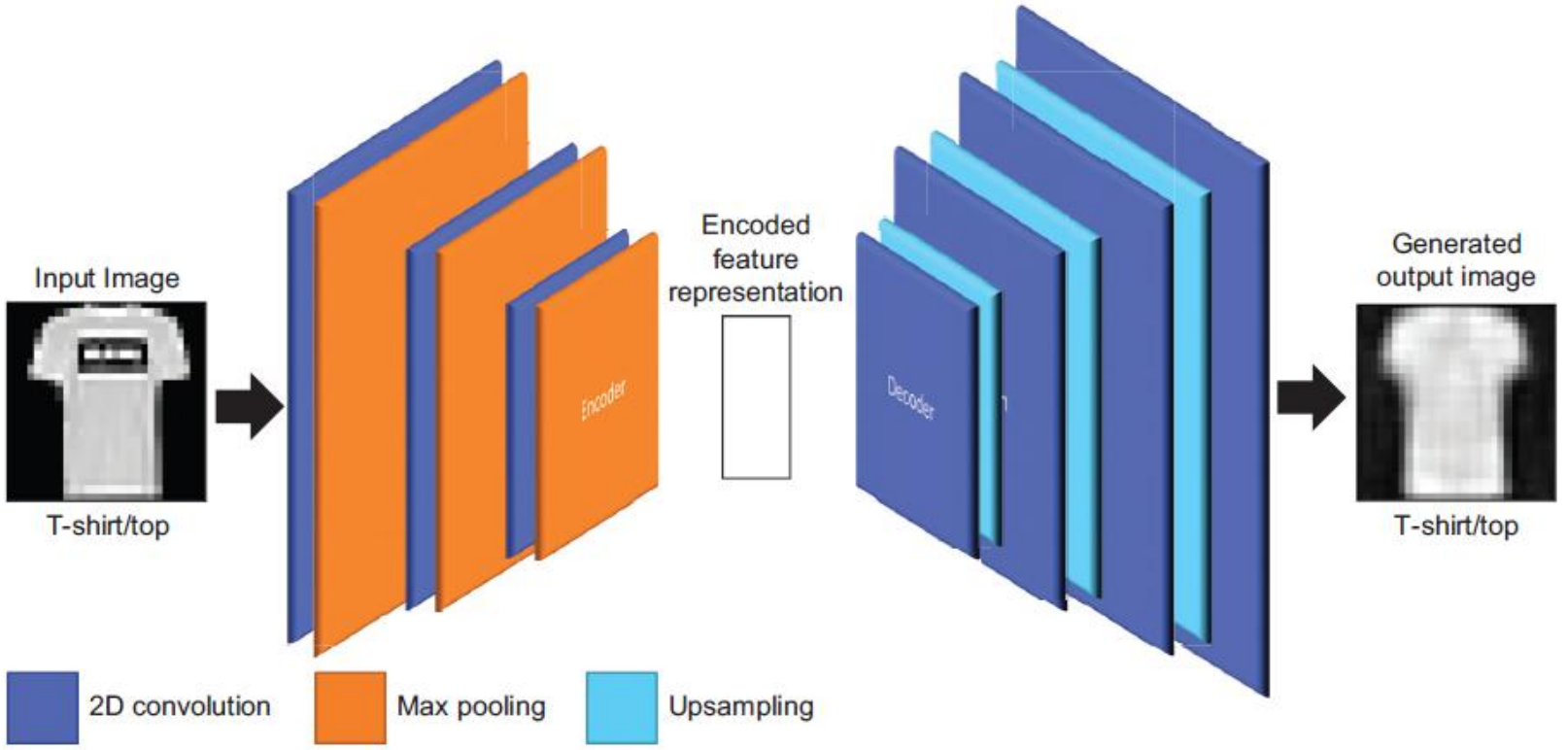
Autoencoder

- Autoencoders take an image as input, create a low-dimensional descriptor from the Image
- This descriptor is often referred to as an embedding of the image and try to reconstruct the input image from the embedding.
- The image embedding is a compressed representation of the image.
- Reconstruction is a lossy process:

  the small, subtle variations in the signal are lost, and only the essential part is retained.
- The loss is the mismatch between the input image and the reconstructed image.
- By minimizing this loss, we incentivize the system to retain the essence of the input as much as possible within the embedding-size budget.

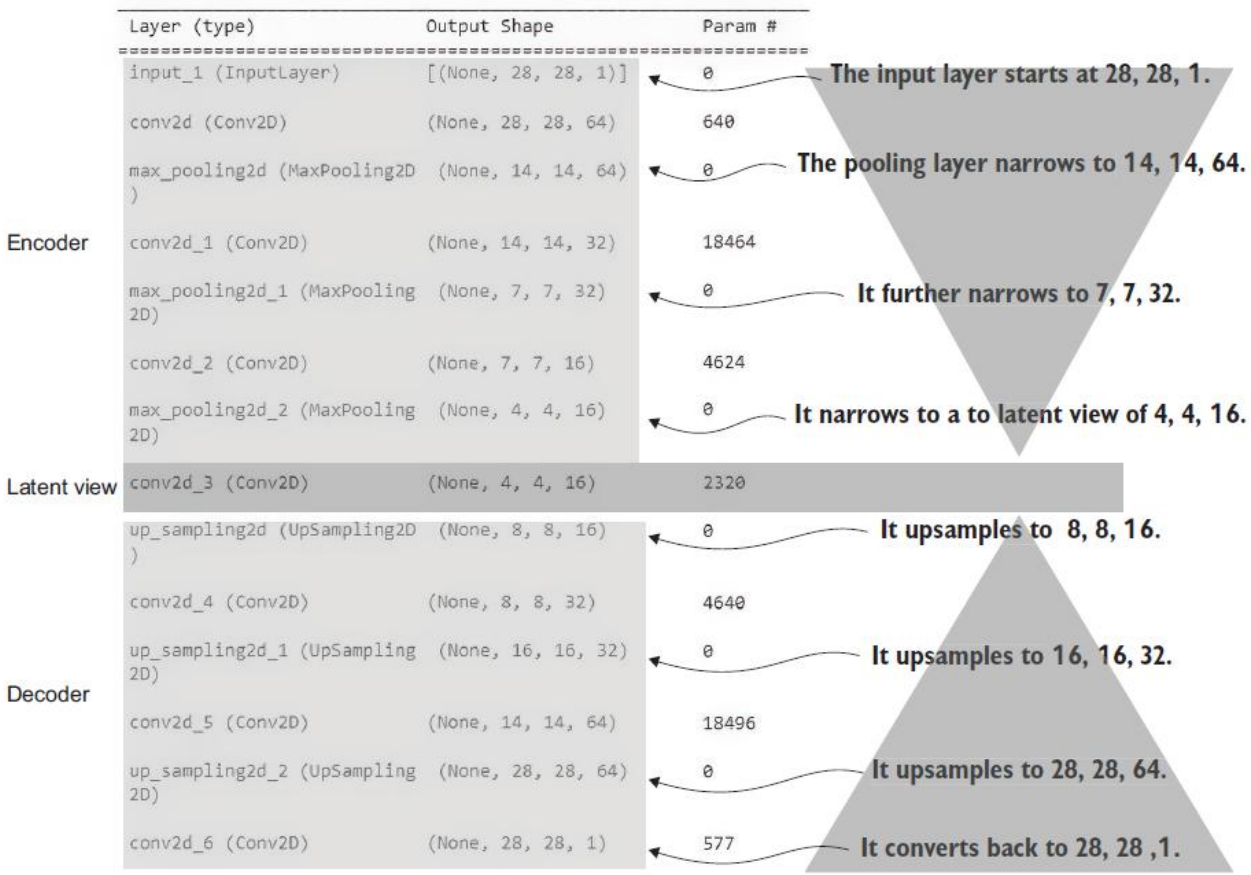
Source: Math and Architectures of Deep Learning, Krishenedu Chauhury

Convolutional Auto-Encoder



Source: Evolutionary Deep Learning, Micheal Lanham

Auto-Encoder model

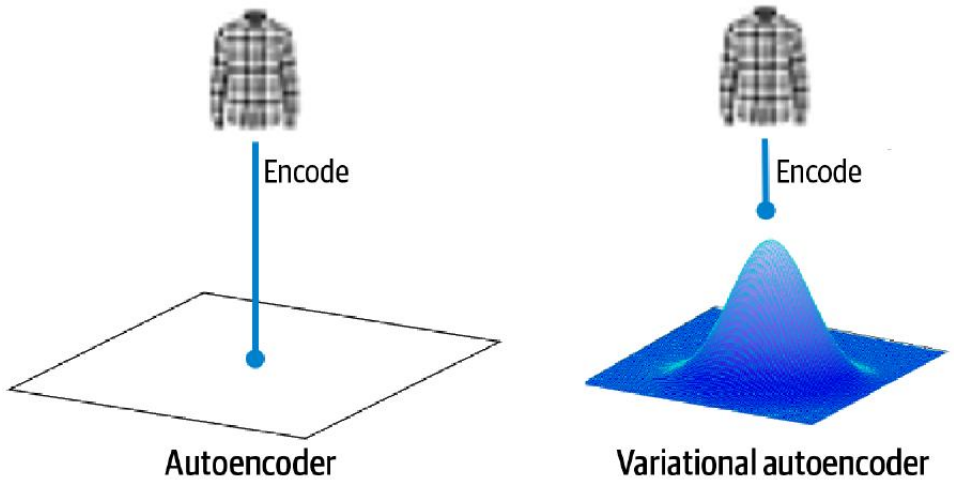


	Layer (type)	Output Shape	Param #	
Encoder	input_1 (InputLayer)	[(None, 28, 28, 1)]	0	The input layer starts at 28, 28, 1.
	conv2d (Conv2D)	(None, 28, 28, 64)	640	
	max_pooling2d (MaxPooling2D)	(None, 14, 14, 64)	0	The pooling layer narrows to 14, 14, 64.
	conv2d_1 (Conv2D)	(None, 14, 14, 32)	18464	
	max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 32)	0	It further narrows to 7, 7, 32.
	conv2d_2 (Conv2D)	(None, 7, 7, 16)	4624	
	max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 16)	0	It narrows to a to latent view of 4, 4, 16.
Latent view	conv2d_3 (Conv2D)	(None, 4, 4, 16)	2320	
Decoder	up_sampling2d (UpSampling2D)	(None, 8, 8, 16)	0	It upsamples to 8, 8, 16.
	conv2d_4 (Conv2D)	(None, 8, 8, 32)	4640	
	up_sampling2d_1 (UpSampling2D)	(None, 16, 16, 32)	0	It upsamples to 16, 16, 32.
	conv2d_5 (Conv2D)	(None, 14, 14, 64)	18496	
	up_sampling2d_2 (UpSampling2D)	(None, 28, 28, 64)	0	It upsamples to 28, 28, 64.
	conv2d_6 (Conv2D)	(None, 28, 28, 1)	577	It converts back to 28, 28 ,1.

Source: Evolutionary Deep Learning, Micheal Lanham

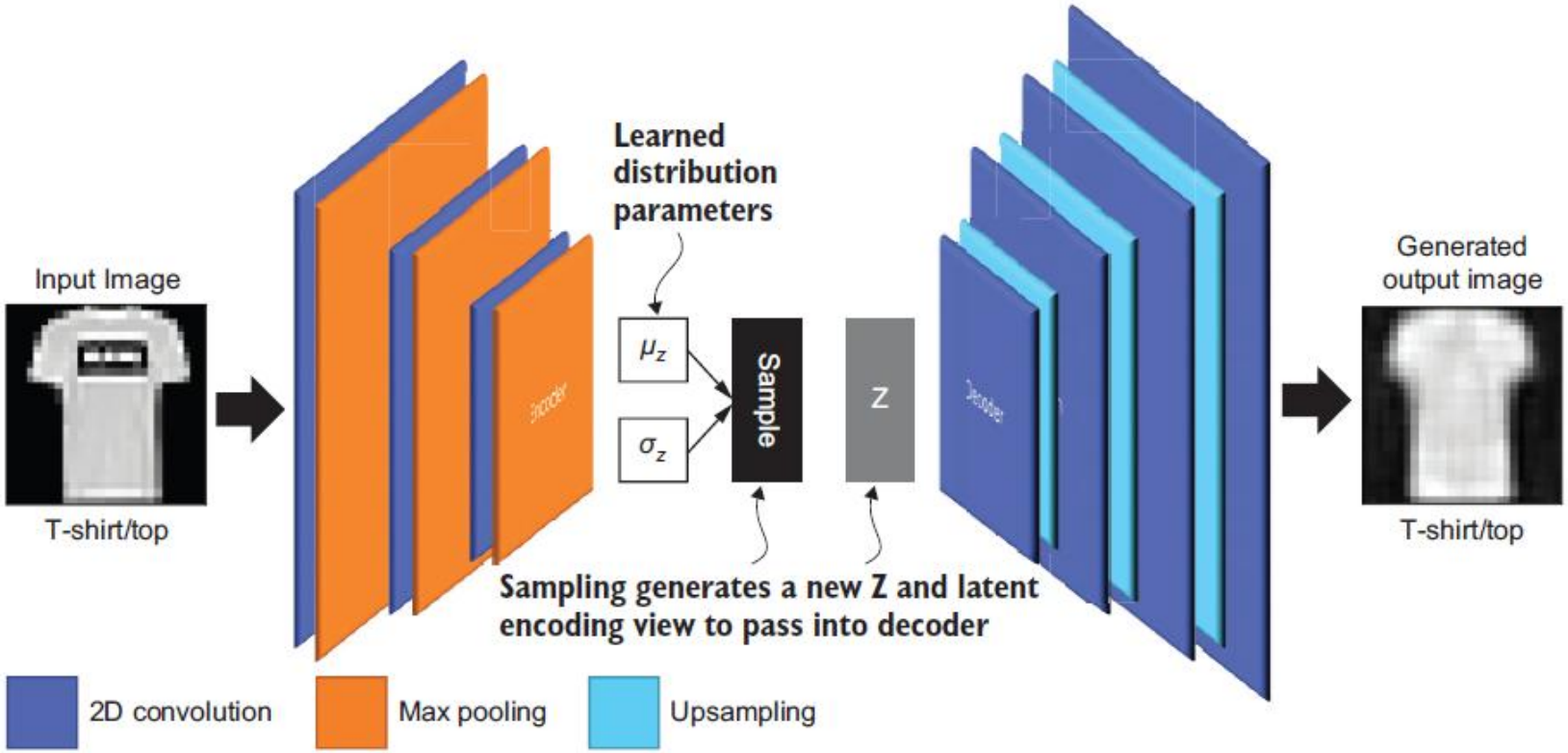
Variational Auto-Encoder (VAE)

- In an auto-encoder, each image is mapped directly to one point in the latent space.
- In a variational auto-encoder, each image is instead mapped to a multivariate normal distribution around a point in the latent space



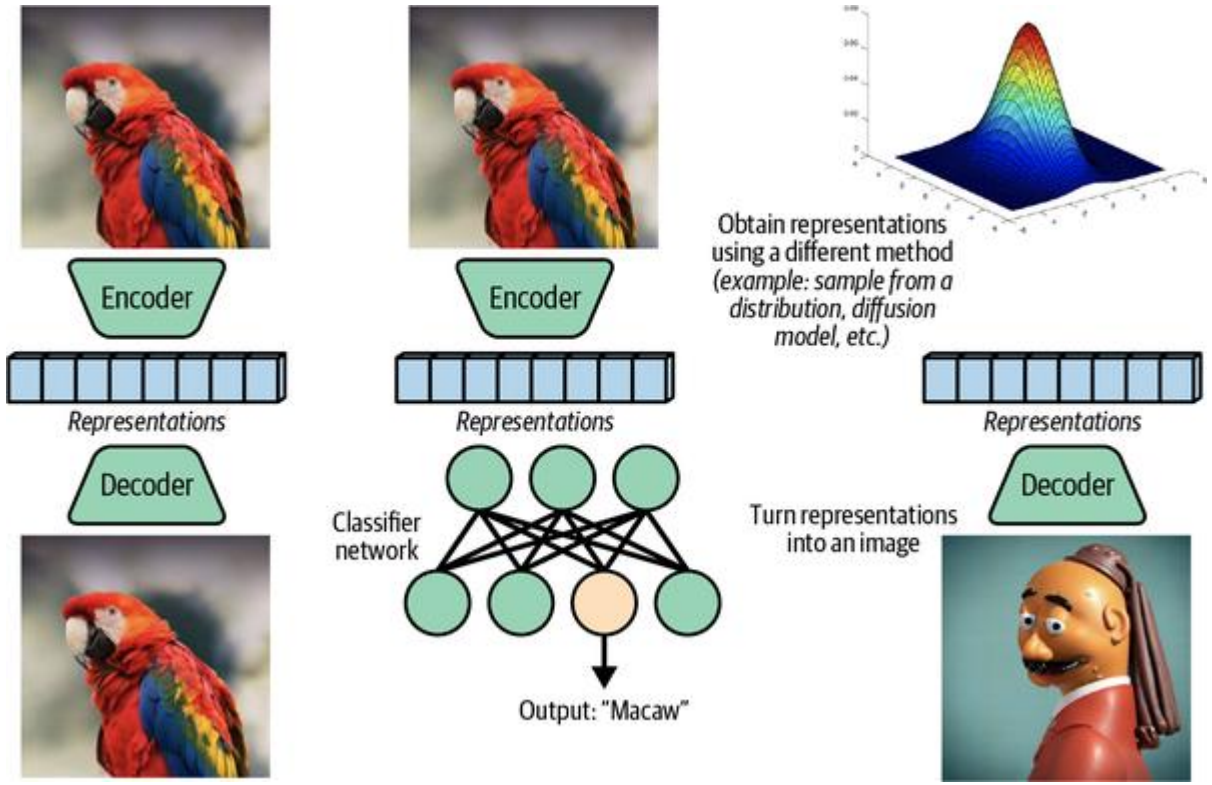
Source: Generative Deep Learning(2nd Ed), David Foster

Variational Auto-Encoder (VAE)



Source: Evolutionary Deep Learning, Micheal Lanham

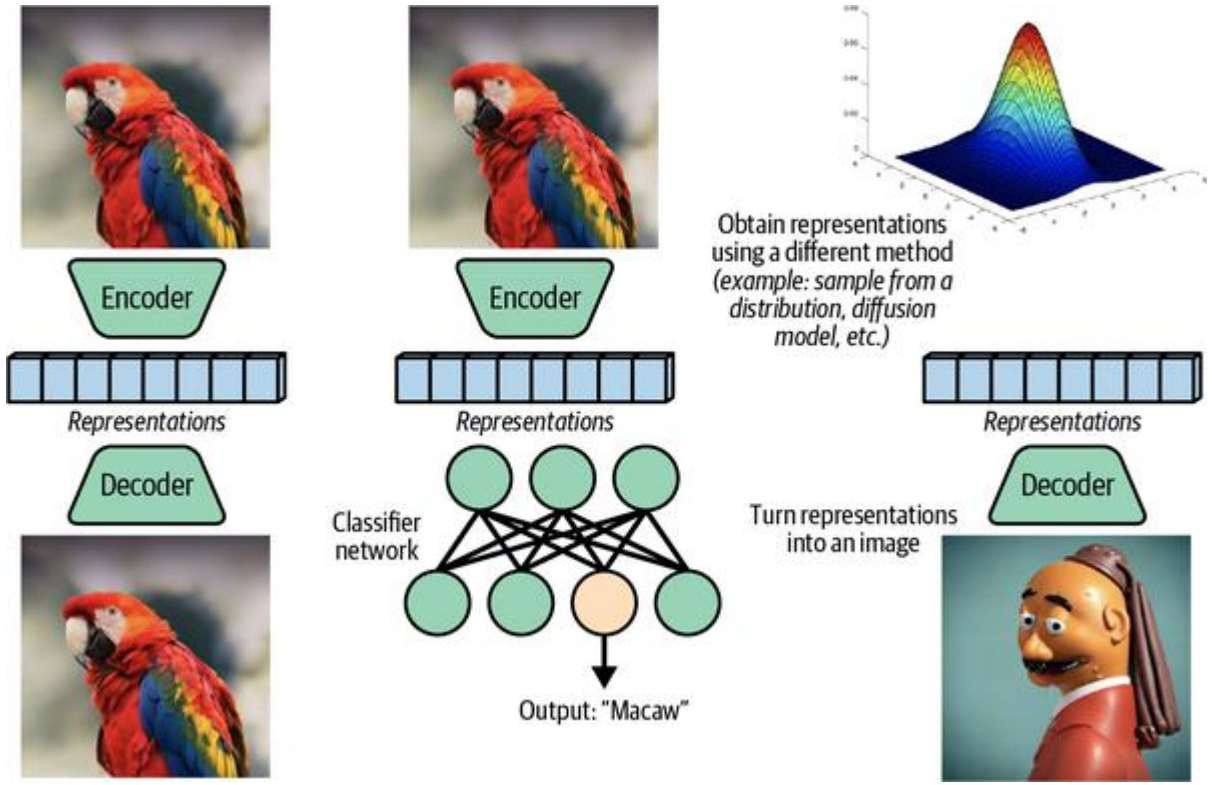
Representing Information



Representations have captured the essential information

Source: Hands-on Generative AI With Transformers and Diffusion Models by Sanseviero, Omar

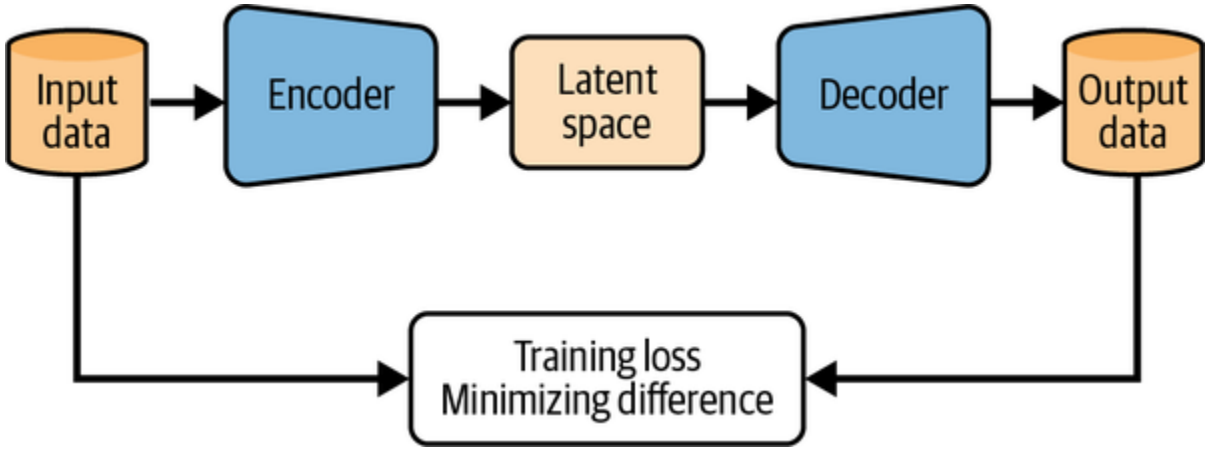
Variational Autoencoders (VAE)



Generate new data by sampling from a random distribution in the latent space

Source: Hands-on Generative AI With Transformers and Diffusion Models by Sanseviero, Omar

Autoencoders



Autoencoder learns to extract key features from the input data

Source: Hands-on Generative AI With Transformers and Diffusion Models by Sanseviero, Omar

Autoencoders/conv block

```
def conv_block(in_channels, out_channels, kernel_size=4, stride=2, padd
    return nn.Sequential(
        nn.Conv2d(
            in_channels,
            out_channels,
            kernel_size=kernel_size,
            stride=stride,
            padding=padding,
        ),
        nn.BatchNorm2d(out_channels),
        nn.ReLU(),
    )
```

Source: Hands-on Generative AI With Transformers and Diffusion Models by Sanseviero, Omar

Autoencoders/Encoder

```
class Encoder(nn.Module):
    def __init__(self, in_channels):
        super().__init__()
        self.conv1 = conv_block(in_channels, 128)
        self.conv2 = conv_block(128, 256)
        self.conv3 = conv_block(256, 512)
        self.conv4 = conv_block(512, 1024)
        self.linear = nn.Linear(1024, 16)

    def forward(self, x):
        x = self.conv1(x)  # (batch size, 128, 14, 14)
        x = self.conv2(x)  # (bs, 256, 7, 7)
        x = self.conv3(x)  # (bs, 512, 3, 3)
        x = self.conv4(x)  # (bs, 1024, 1, 1)
        # Keep batch dimension when flattening
        x = self.linear(x.flatten(start_dim=1))  # (bs, 16)
        return x
```

Source: Hands-on Generative AI With Transformers and Diffusion Models by Sanseviero, Omar

Autoencoders conv_transpose

```python
def conv_transpose_block(
    in_channels,
    out_channels,
    kernel_size=3,
    stride=2,
    padding=1,
    output_padding=0,
    with_act=True,
):
    modules = [
        nn.ConvTranspose2d(
            in_channels,
            out_channels,
            kernel_size=kernel_size,
            stride=stride,
            padding=padding,
            output_padding=output_padding,
        ),
    ]
    if with_act:  # Controlling this will be handy later
        modules.append(nn.BatchNorm2d(out_channels))
        modules.append(nn.ReLU())
    return nn.Sequential(*modules)
```

Source: Hands-on Generative AI With Transformers and Diffusion Models by Sanseviero, Omar

Autoencoders/Decoder

```
class Decoder(nn.Module):
    def __init__(self, out_channels):
        super().__init__()

        self.linear = nn.Linear(
            16, 1024 * 4 * 4
        )  # note it's reshaped in forward
        self.t_conv1 = conv_transpose_block(1024, 512)
        self.t_conv2 = conv_transpose_block(512, 256, output_padding=1)
        self.t_conv3 = conv_transpose_block(256, out_channels, output_p

    def forward(self, x):
        bs = x.shape[0]
        x = self.linear(x)  # (bs, 1024*4*4)
        x = x.reshape((bs, 1024, 4, 4))  # (bs, 1024, 4, 4)
        x = self.t_conv1(x)  # (bs, 512, 7, 7)
        x = self.t_conv2(x)  # (bs, 256, 14, 14)
        x = self.t_conv3(x)  # (bs, 1, 28, 28)
        return x
```

Source: Hands-on Generative AI With Transformers and Diffusion Models by Sanseviero, Omar

Autoencoder

```python
class AutoEncoder(nn.Module):
    def __init__(self, in_channels):
        super().__init__()
        self.encoder = Encoder(in_channels)
        self.decoder = Decoder(in_channels)

    def encode(self, x):
        return self.encoder(x)

    def decode(self, x):
        return self.decoder(x)

    def forward(self, x):
        return self.decode(self.encode(x))
```

```python
model = AutoEncoder(1)
```

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 128, 14, 14]	2,176
BatchNorm2d-2	[-1, 128, 14, 14]	256
ReLU-3	[-1, 128, 14, 14]	0
Conv2d-4	[-1, 256, 7, 7]	524,544
BatchNorm2d-5	[-1, 256, 7, 7]	512
ReLU-6	[-1, 256, 7, 7]	0
Conv2d-7	[-1, 512, 3, 3]	2,097,664
BatchNorm2d-8	[-1, 512, 3, 3]	1,024
ReLU-9	[-1, 512, 3, 3]	0
Conv2d-10	[-1, 1024, 1, 1]	8,389,632
BatchNorm2d-11	[-1, 1024, 1, 1]	2,048
ReLU-12	[-1, 1024, 1, 1]	0
Linear-13	[-1, 16]	16,400
Encoder-14	[-1, 16]	0
Linear-15	[-1, 16384]	278,528
ConvTranspose2d-16	[-1, 512, 7, 7]	4,719,104
BatchNorm2d-17	[-1, 512, 7, 7]	1,024
ReLU-18	[-1, 512, 7, 7]	0
ConvTranspose2d-19	[-1, 256, 14, 14]	1,179,904
BatchNorm2d-20	[-1, 256, 14, 14]	512
ReLU-21	[-1, 256, 14, 14]	0
ConvTranspose2d-22	[-1, 1, 28, 28]	2,305
BatchNorm2d-23	[-1, 1, 28, 28]	2
ReLU-24	[-1, 1, 28, 28]	0
Decoder-25	[-1, 1, 28, 28]	0

Total params: 17,215,635

Source: Hands-on Generative AI With Transformers and Diffusion Models by Sanseviero, Om

Autoencoder/train

loss function (MSE):
Measures the difference between the reconstructed and original images in pixel level.

$$MSE = \frac{1}{m}\sum_{i=1}^{m}(\widehat{y^i} - y^i)^2$$

```
num_epochs = 10
lr = 1e-4

device = get_device()
model = model.to(device)
optimizer = torch.optim.AdamW(model.parameters(), lr=lr, eps=1e-5)

losses = []  # List to store the loss values for plotting
for _ in (progress := trange(num_epochs, desc="Training")):
    for _, batch in (
        inner := tqdm(enumerate(train_dataloader), total=len(train_da
    ):
        batch = batch.to(device)

        # Pass through the model and obtain reconstructed images
        preds = model(batch)

        # Compare the prediction with the original images
        loss = F.mse_loss(preds, batch)

        # Display loss and store for plotting
        inner.set_postfix(loss=f"{loss.cpu().item():.3f}")
        losses.append(loss.item())

        # Update the model parameters with the optimizer based on thi
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()
    progress.set_postfix(loss=f"{loss.cpu().item():.3f}", lr=f"{lr:.6
```

Source: Hands-on Generative AI With Transformers and Diffusion Models by S

Latent space in Autoencoder

representations of similar inputs are usually clustered close together,
but there's a significant amount of overlap and empty space,
as well as substantial variability in the amount of latent space dedicated to each class.
If we choose a random point in the latent space and pass it through the decoder,
we can't faithfully predict what result will come up.

Source: Hands-on Generative AI With Transformers and Diffusion Models by Sanseviero, Omar

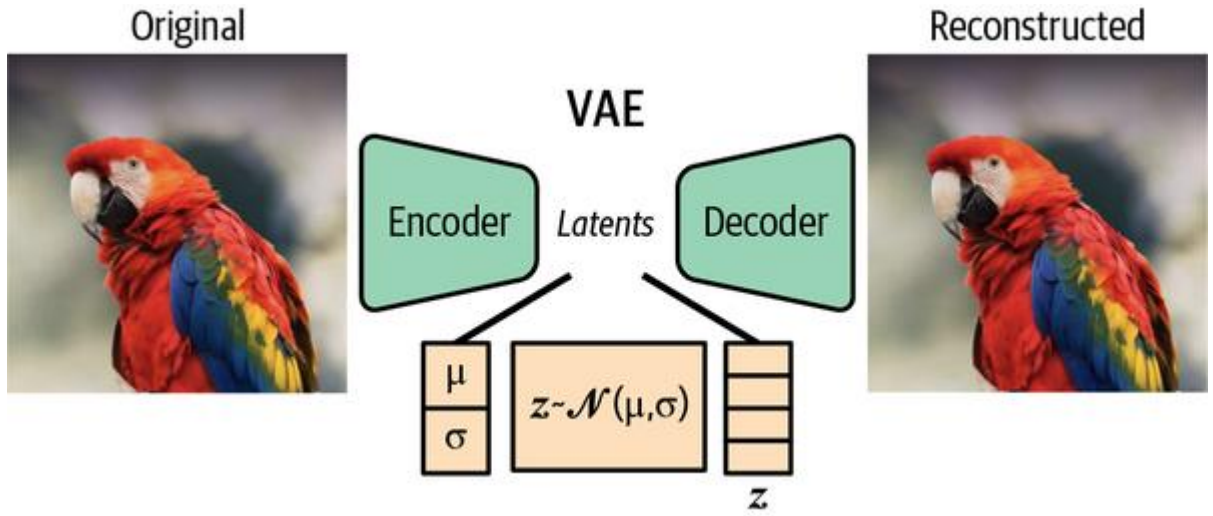
Variational Autoencoder

VAEs learn a probability distribution for each feature in the latent space.

Instead of mapping inputs to specific points,

VAEs represent each feature with a Gaussian distribution,

capturing the variability of that feature within the data

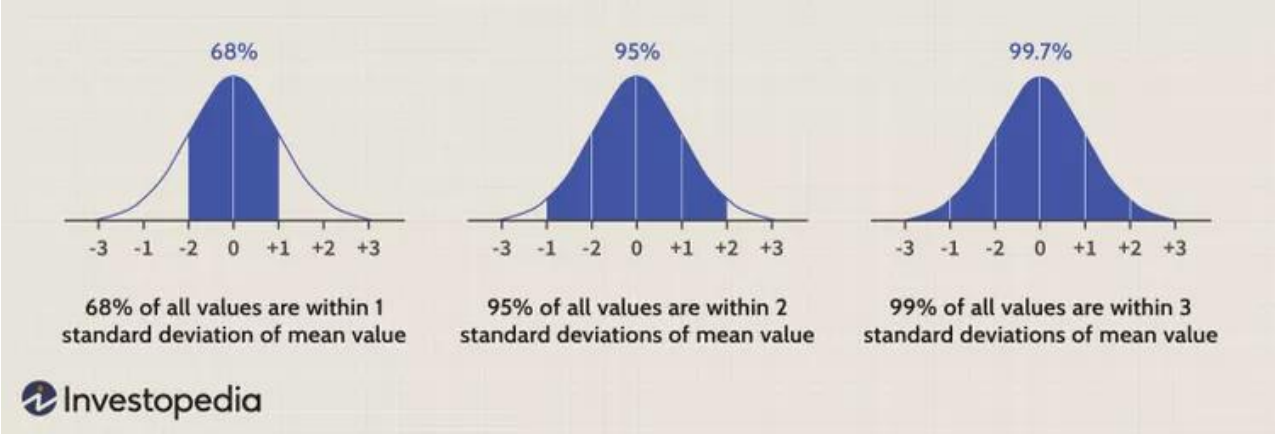


Source: Hands-on Generative AI With Transformers and Diffusion Models by Sanseviero, Omar

Normal distribution (or Gaussian distribution)

with a mean μ, and standard deviation σ

$$g(x, \mu, \sigma) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$



$$var = \sigma^2$$

$$\log(var) = 2\log(\sigma)$$

$$\log(\sigma) = \frac{1}{2}\log(var)$$

$$\sigma = \exp(\frac{1}{2}\log var)$$

Source: https://www.investopedia.com/terms/n/normaldistribution.asp

VAE/encoder

```python
class VAEEncoder(nn.Module):
    def __init__(self, in_channels, latent_dims):
        super().__init__()

        self.conv_layers = nn.Sequential(
            conv_block(in_channels, 128),
            conv_block(128, 256),
            conv_block(256, 512),
            conv_block(512, 1024),
        )

        # Define fully connected layers for mean and log-variance
        self.mu = nn.Linear(1024, latent_dims)
        self.logvar = nn.Linear(1024, latent_dims)

    def forward(self, x):
        bs = x.shape[0]
        x = self.conv_layers(x)
        x = x.reshape(bs, -1)
        mu = self.mu(x)
        logvar = self.logvar(x)
        return (mu, logvar)
```

- mu represents the mean,
- logvar represents the logarithm of the variance

Source: Hands-on Generative AI With Transformers and Diffusion Models by Sanseviero, Omar

VAE/reconstructed

$$\sigma = \exp(\frac{1}{2}\log var)$$

- Reparameterization trick
- Instead of regular definition of *z*:

$$z \sim N(\mu, \sigma^2)$$

- We define *z* as:

$$\boldsymbol{z = \mu + \sigma \cdot \epsilon}$$

$$\epsilon \sim N(0,1)$$

$$z = g(\mu, \sigma, \epsilon)$$

Source: https://dilithjay.com/blog/the-reparameterization-trick-clearly-explained

```python
class VAE(nn.Module):
    def __init__(self, in_channels, latent_dims):
        super().__init__()
        self.encoder = VAEEncoder(in_channels, latent_dims) ❶
        self.decoder = Decoder(in_channels, latent_dims)

    def encode(self, x):
        # Returns mu, log_var
        return self.encoder(x)

    def decode(self, z):
        return self.decoder(z)

    def forward(self, x):
        # Obtain parameters of the normal (Gaussian) distribution
        mu, logvar = self.encode(x) ❷

        # Sample from the distribution
        std = torch.exp(0.5 * logvar) ❸
        z = self.sample(mu, std) ❹

        # Decode the latent point to pixel space
        reconstructed = self.decode(z) ❺

        # Return the reconstructed image, and also the mu and logvar
        # so we can compute a distribution loss
        return reconstructed, mu, logvar ❻

    def sample(self, mu, std):
        # Reparametrization trick
        # Sample from N(0, I), translate and scale
        eps = torch.randn_like(std) ❼
        return mu + eps * std
```

VAE/loss

- Reconstruction loss

Measures how much the output images resemble the originals

- KLD

Measures how well the features follow a Gaussian distribution

- Total loss

The addition of the two previous losses

```python
def vae_loss(batch, reconstructed, mu, logvar):
    bs = batch.shape[0]

    # Reconstruction loss from the pixels - 1 per image
    reconstruction_loss = F.mse_loss(
        reconstructed.reshape(bs, -1),
        batch.reshape(bs, -1),
        reduction="none",
    ).sum(dim=-1)

    # KL-divergence loss, per input image
    kl_loss = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp(),

    # Combine both losses and get the mean across images
    loss = (reconstruction_loss + kl_loss).mean(dim=0)

    return (loss, reconstruction_loss, kl_loss)
```

Source: Hands-on Generative AI With Transformers and Diffusion Models by Sanseviero, Omar

VAE/KLD loss

- we want the features to follow a Gaussian distribution.

- Kullback–Leibler divergence, or KL divergence (KLD):
  measure how much a probability distribution differs from another one.

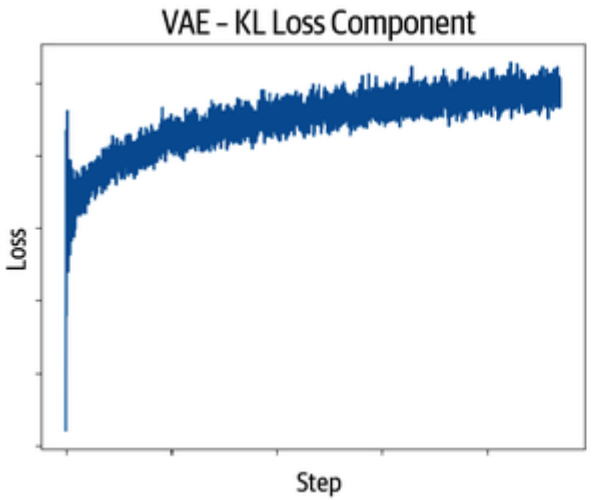
In the case of multivariate isotropic Gaussian distributions:

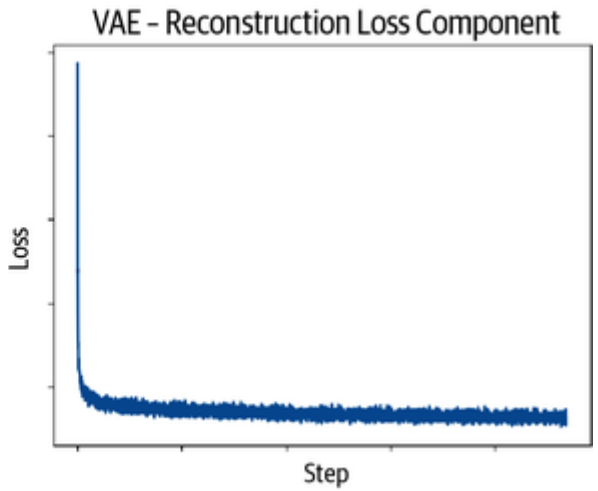
$$D_{KL}[\mathcal{N}(\mu, \sigma^2) || \mathcal{N}(0,1)] = -\frac{1}{2} \sum (1 + log(\sigma^2) - \mu^2 - \sigma^2)$$

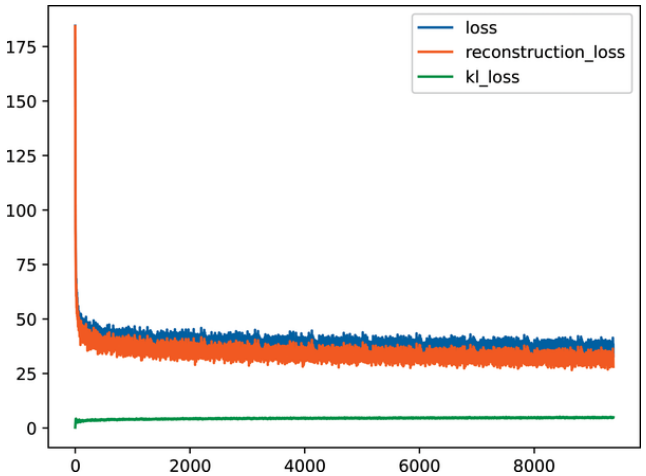
```
kl_loss = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp(), dim=-1)
```

Source: Hands-on Generative AI With Transformers and Diffusion Models by Sanseviero, Omar

VAE/train







Source: Hands-on Generative AI With Transformers and Diffusion Models by Sanseviero, Omar

Topics(2)

- Generative Adversarial Networks (GAN)

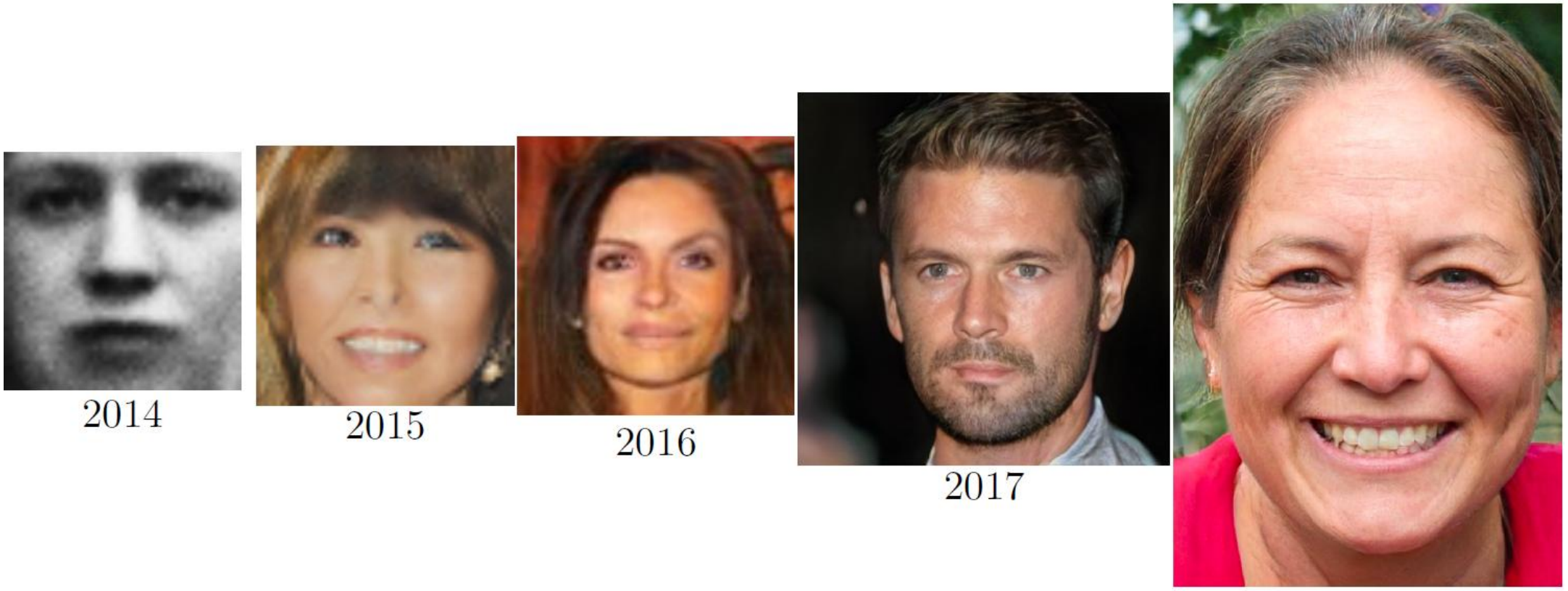
Sample Generation using GAN

Training Data
(CelebA)

Sample Generator
(Karras et al, 2017)

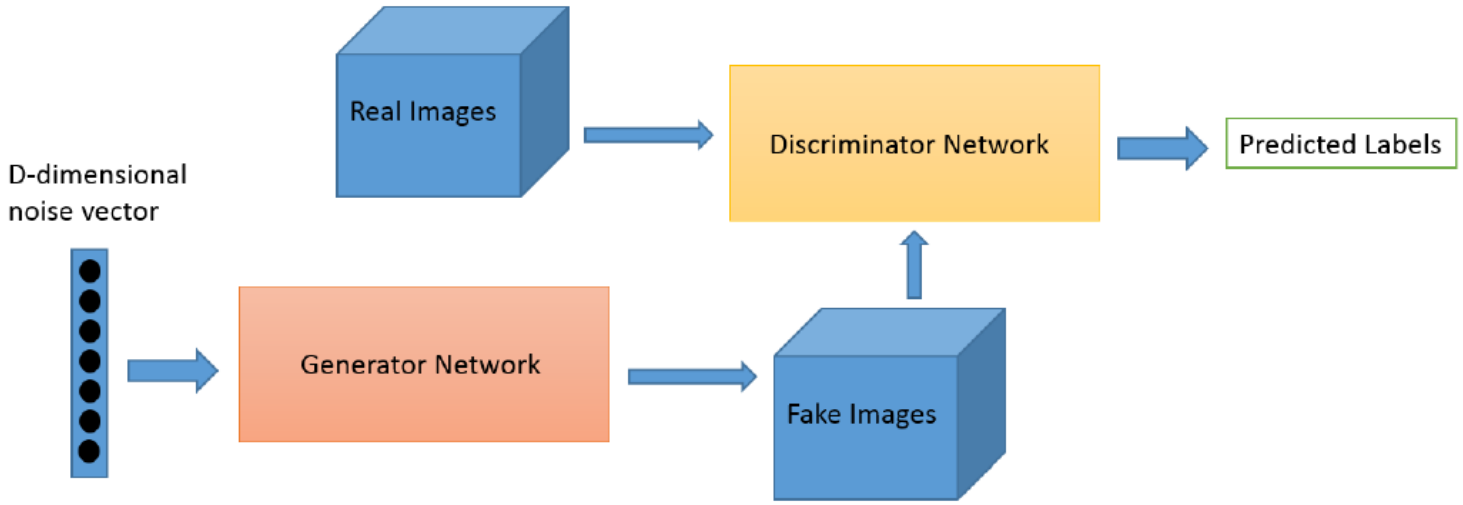
Source: “Adversarial Machine Learning,” by Ian Goodfellow, ICLR Keynote, 2019

Progress of faces in GAN



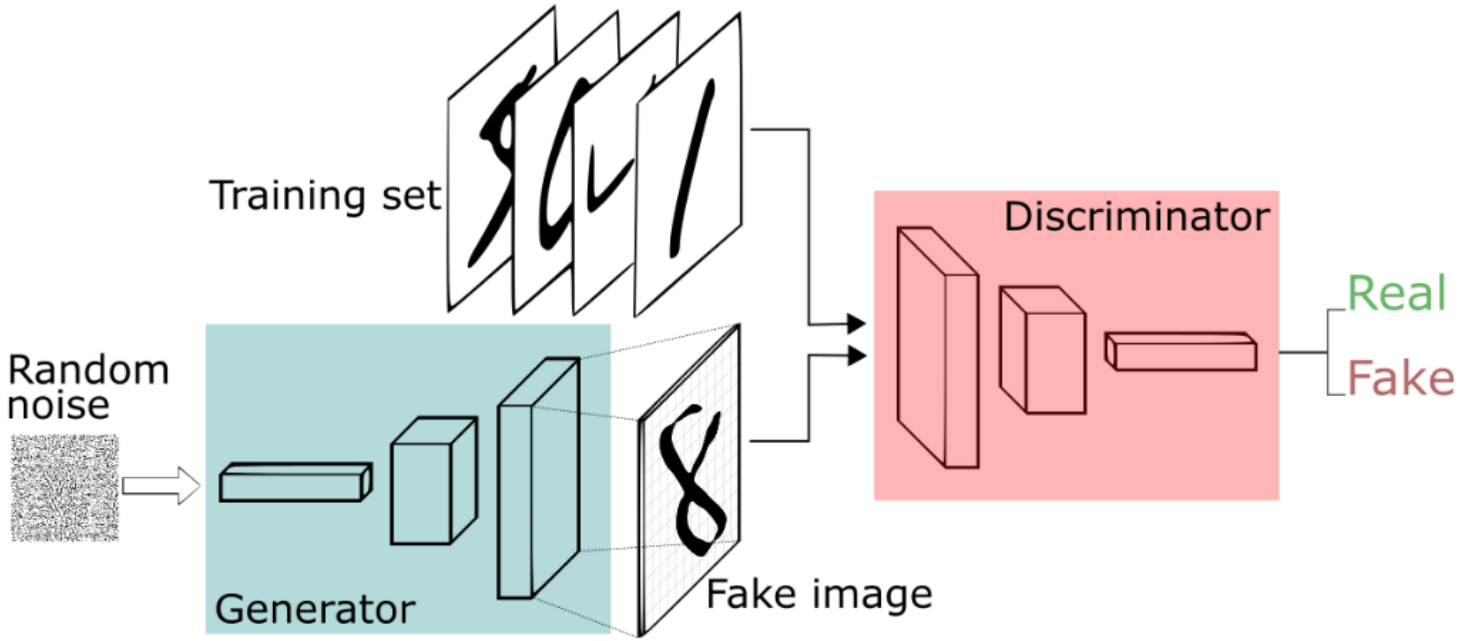
Source: “Adversarial Machine Learning,” by Ian Goodfellow, ICLR Keynote, 2019

Generative Adversarial Networks (GAN)



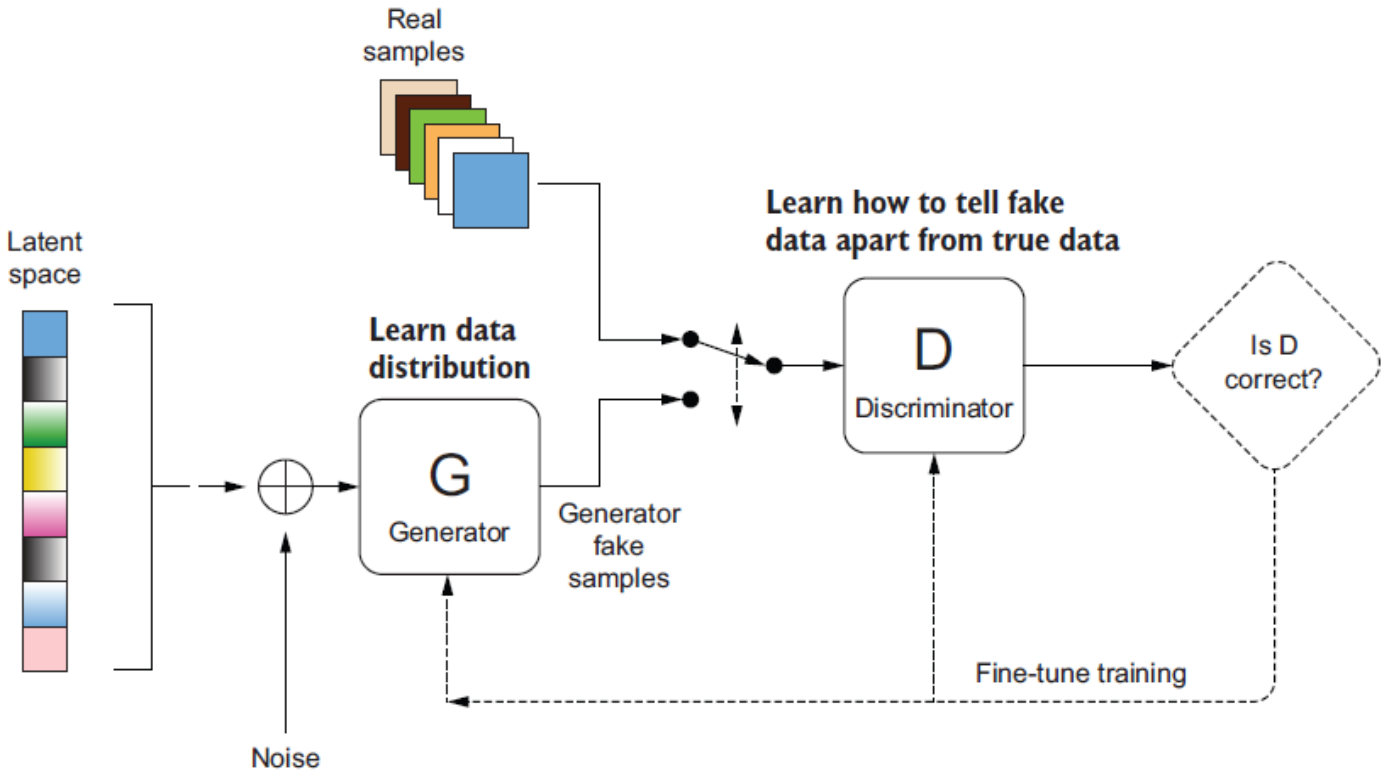
Source: Generative Adversarial Networks for beginners, By Jon BrunerAdit Deshpande, 2017
O'Reilly Media

Generative Adversarial Networks (GAN)



Source: https://www.freecodecamp.org/news/an-intuitive-introduction-to-generative-adversarial-networks-gans-7a2264a81394/

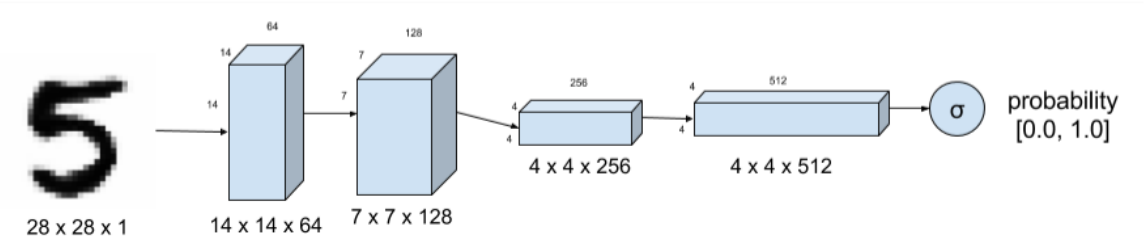
Generative Adversarial Networks (GAN)



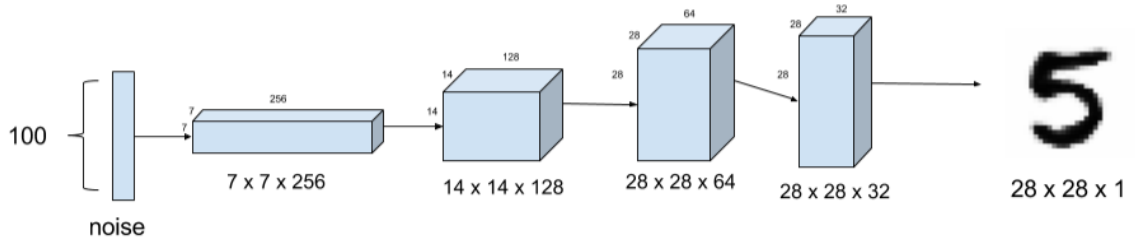
Source: Evolutionary Deep Learning, Micheal Lanham

Vallina GAN

Discriminator



Generator



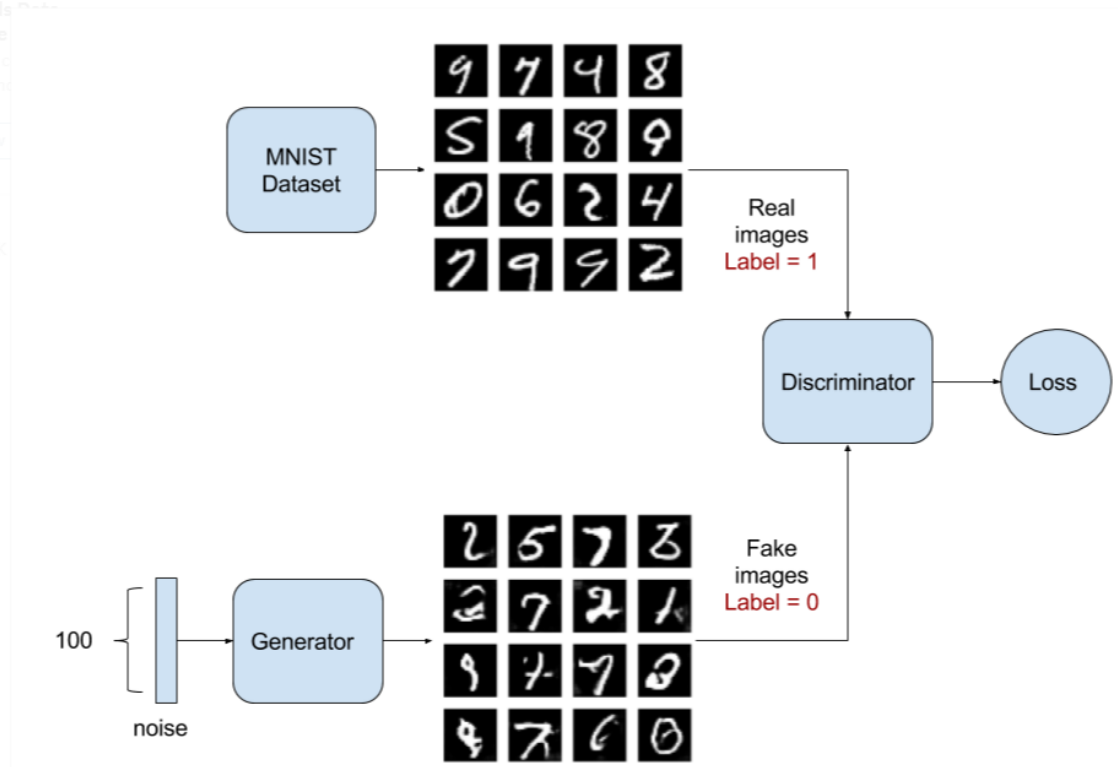
Adversarial Model

Source: https://towardsdatascience.com/gan-by-example-using-keras-on-tensorflow-backend-1a6d515a60d0

SJSU SAN JOSÉ STATE UNIVERSITY

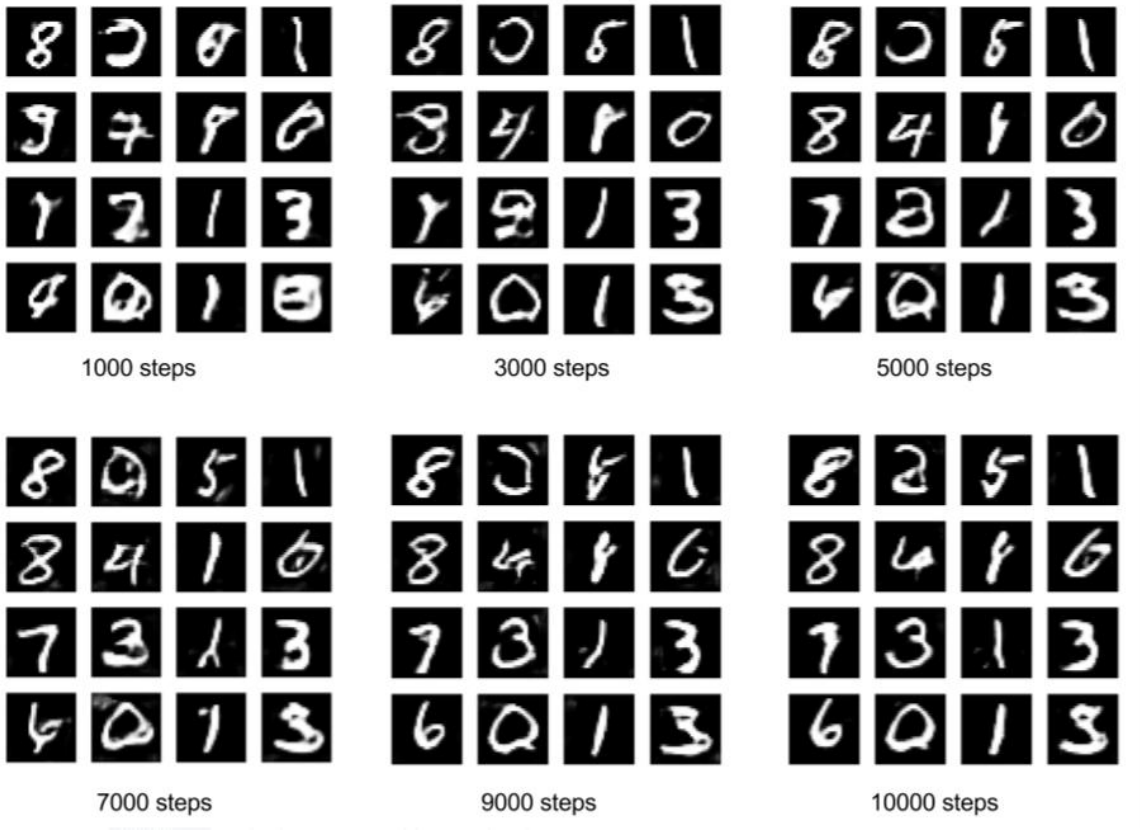
Vallina GAN

Training



Source: https://towardsdatascience.com/gan-by-example-using-keras-on-tensorflow-backend-1a6d515a60d0

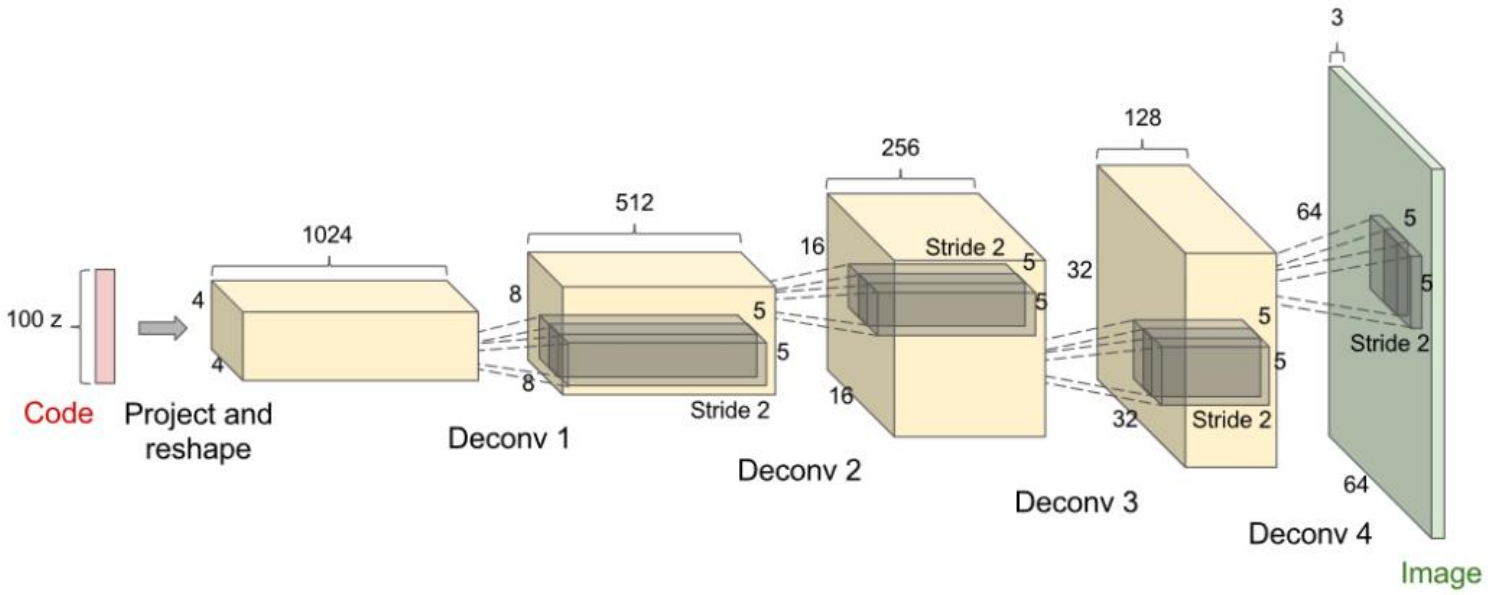
Vallina GAN



Source: https://towardsdatascience.com/gan-by-example-using-keras-on-tensorflow-backend-1a6d515a60d0

Generator

DCGAN (deep convolutional GAN)



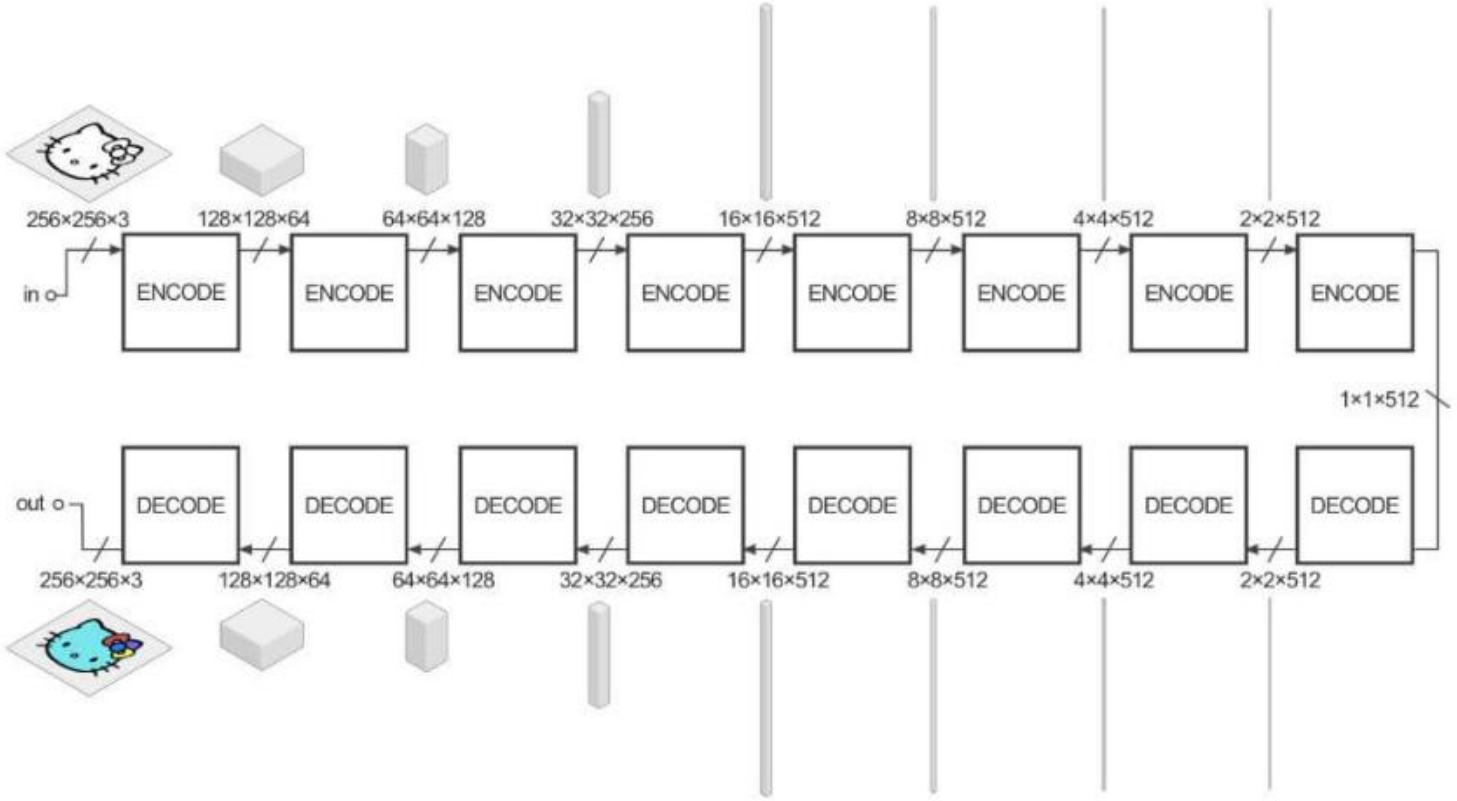
Source: Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks
Alec Radford, Luke Metz, Soumith Chintala

Generator

U-Net

encoder: C64-C128-C256-C512-C512-C512-C512-C512

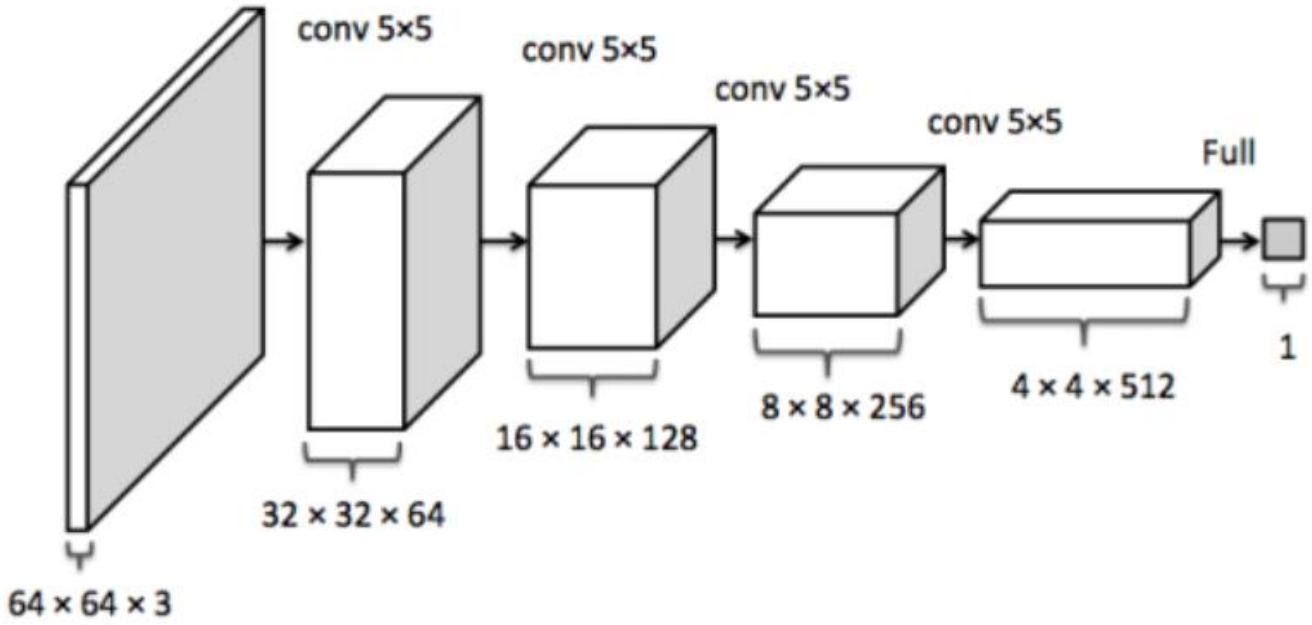
decoder: CD512-CD512-CD512-C512-C256-C128-C64



Source: https://neurohive.io/en/popular-networks/pix2pix-image-to-image-translation/

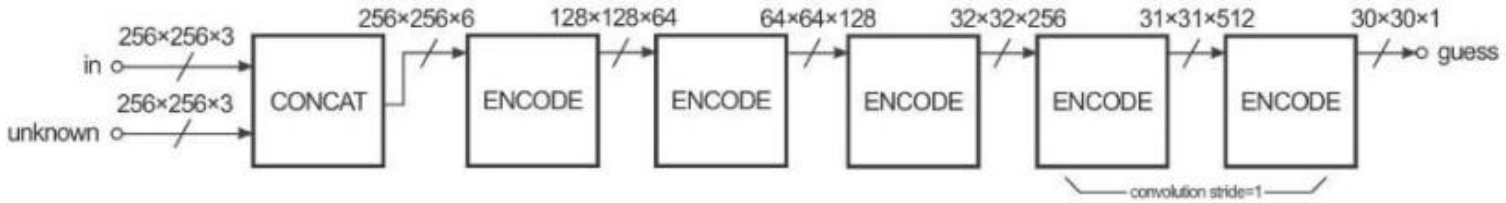
SJSU SAN JOSÉ STATE UNIVERSITY

Discriminator



Discriminator

PatchGAN



1x1 discriminator: C64-C128 (convolutions are 1x1 spatial filters)
16x16 discriminator: C64-C128
70x70 discriminator: C64-C128-C256-C512
286x286 discriminator: C64-C128-C256-C512-C512-C512

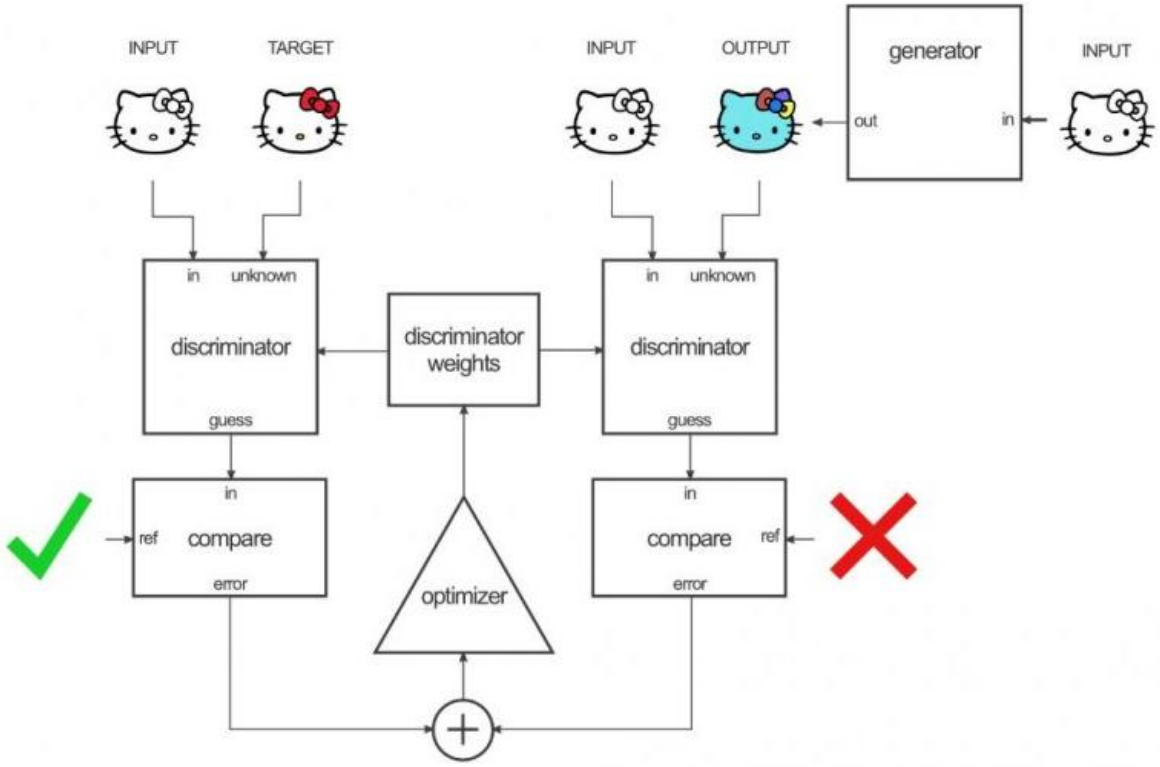
Source: https://neurohive.io/en/popular-networks/pix2pix-image-to-image-translation/

Generator and Discriminator in GAN

	Generator	Discriminator
Input	A vector of random numbers	The Discriminator receives input from two sources: ■ Real examples coming from the training dataset ■ Fake examples coming from the Generator
Output	Fake examples that strive to be as convincing as possible	Predicted probability that the input example is real
Goal	Generate fake data that is indistinguishable from members of the training dataset	Distinguish between the fake examples coming from the Generator and the real examples coming from the training dataset

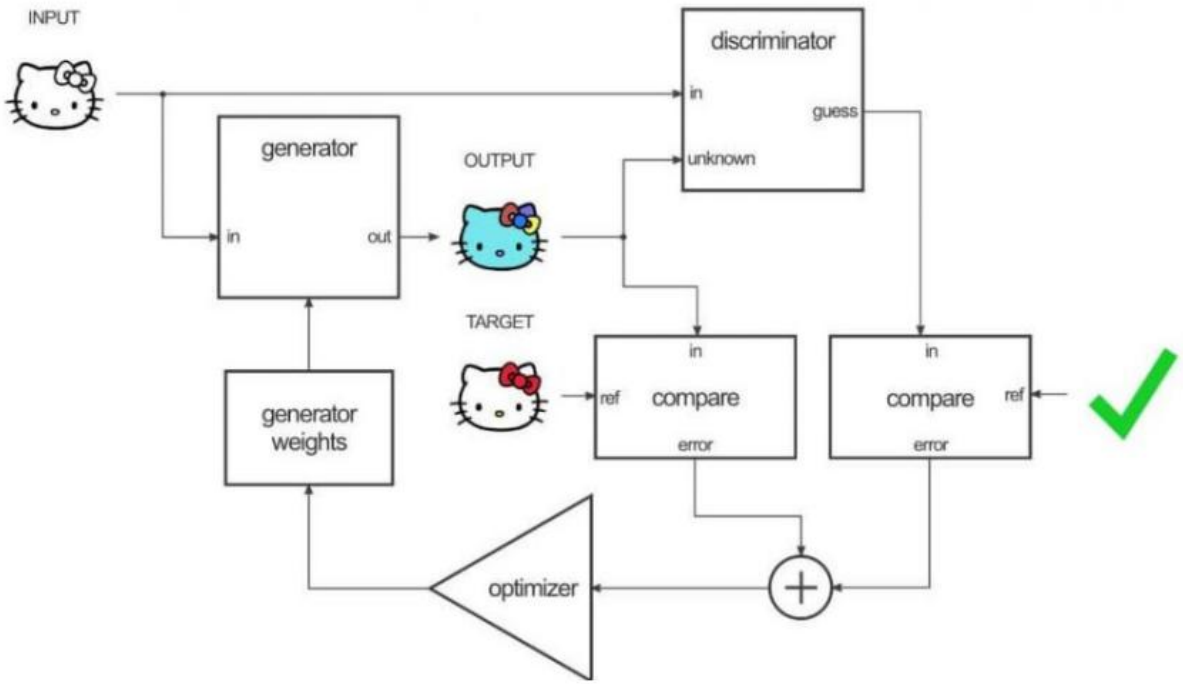
Source: GANs in action, Jakub Langr & Vladimir Bok

Training discriminator



Source: https://neurohive.io/en/popular-networks/pix2pix-image-to-image-translation/

Training generator



Source: https://neurohive.io/en/popular-networks/pix2pix-image-to-image-translation/

Conflicting objectives

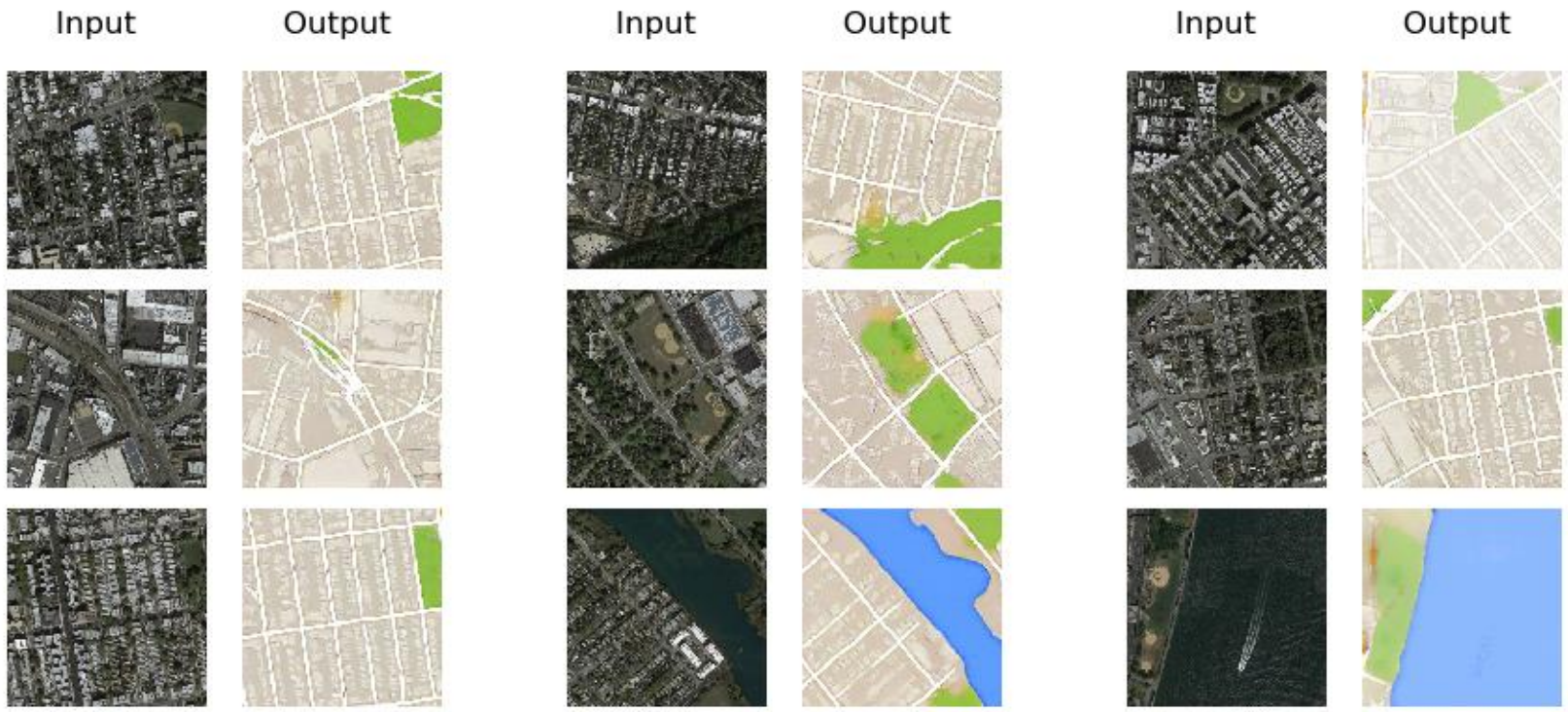
- The Discriminator's goal is to be as accurate as possible.
- The Generator seeks to fool the Discriminator by producing fake examples that are indistinguishable from the real data.

Source: GANs in action, Jakub Langr & Vladimir Bok

Train GAN

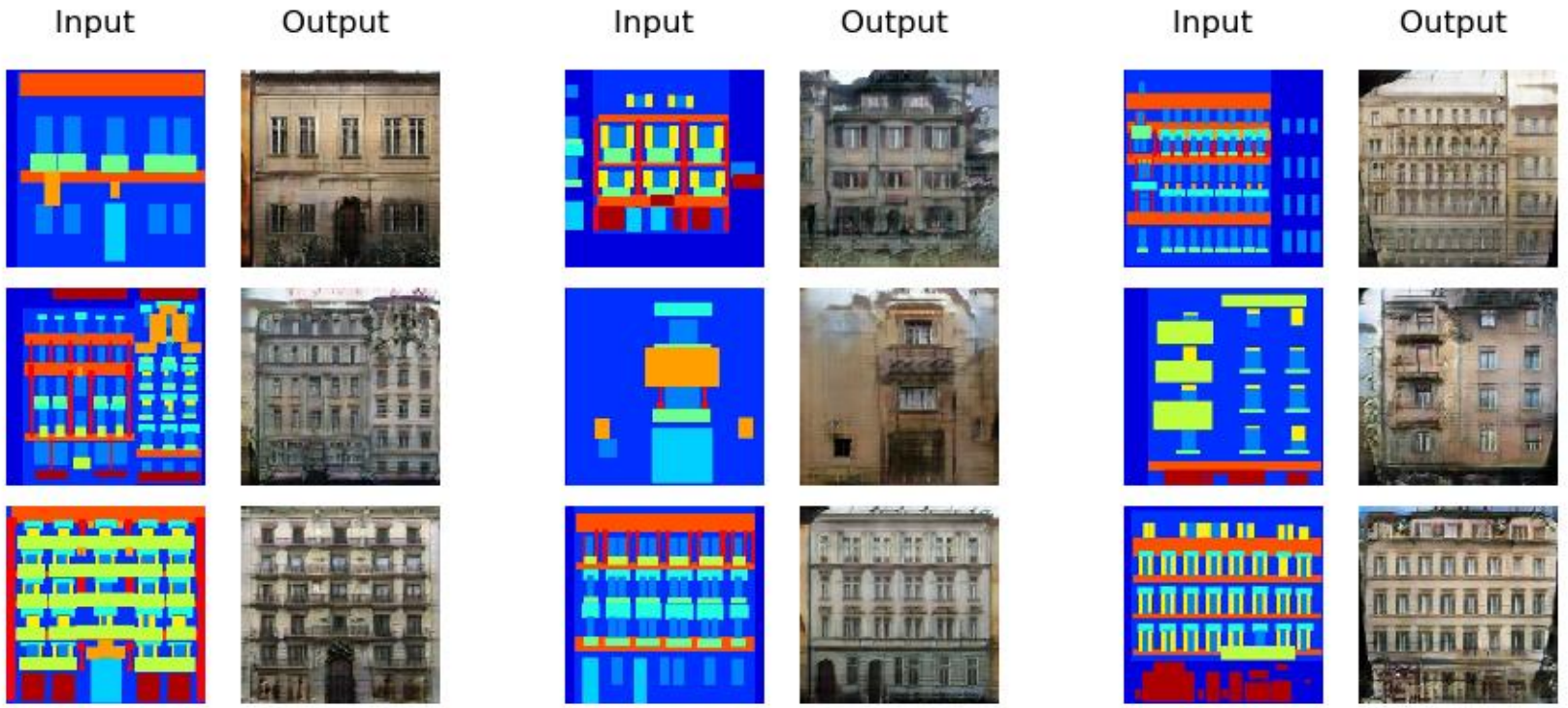
Step 1: Set a task and dataset
Step 2: Define architecture of GAN
Step 3: Train Generator to produce fake data
Step 4: With fake data and real data, train Discriminator
Step 5: Train generator with the output of discriminator
Step 6: Repeat step 4 & 5

Maps_AtoB

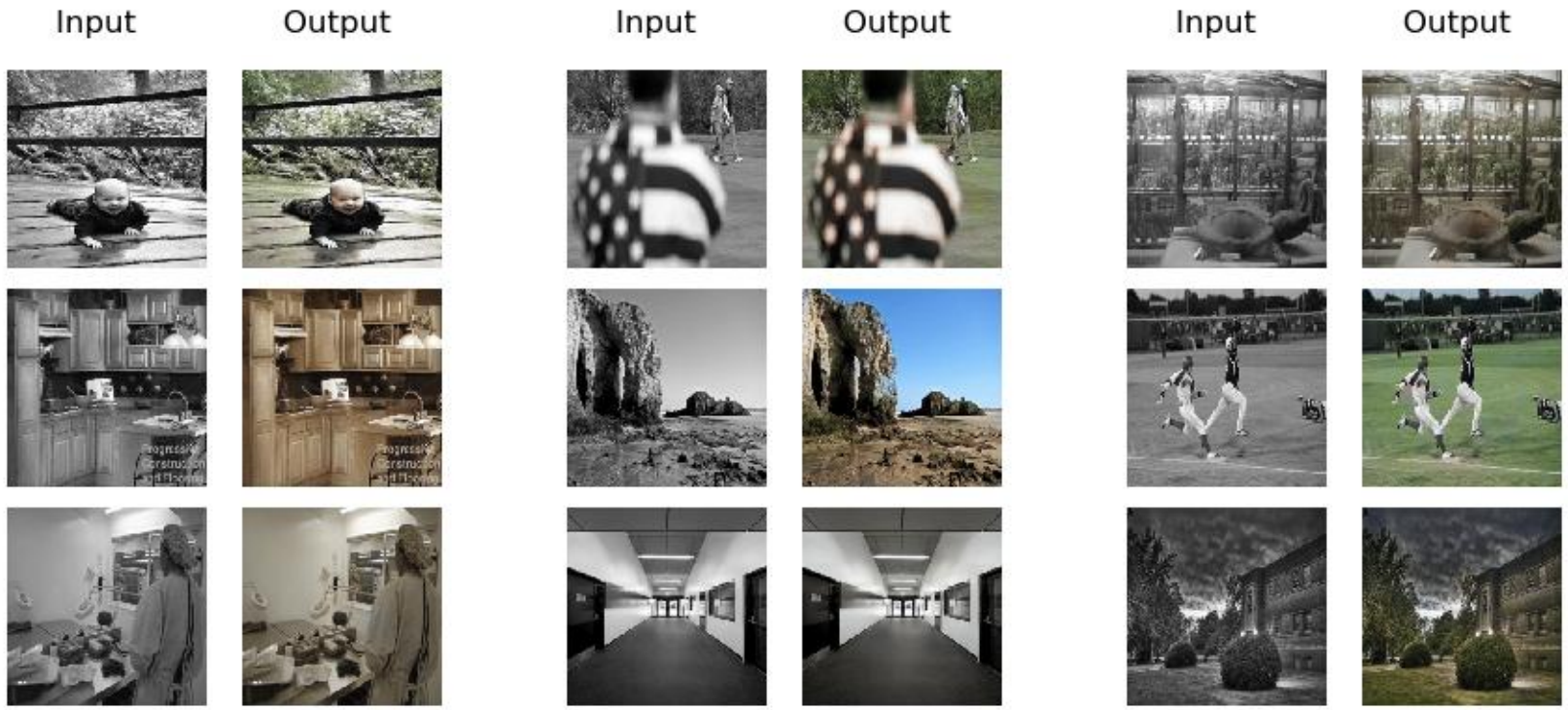


Maps_BtoA

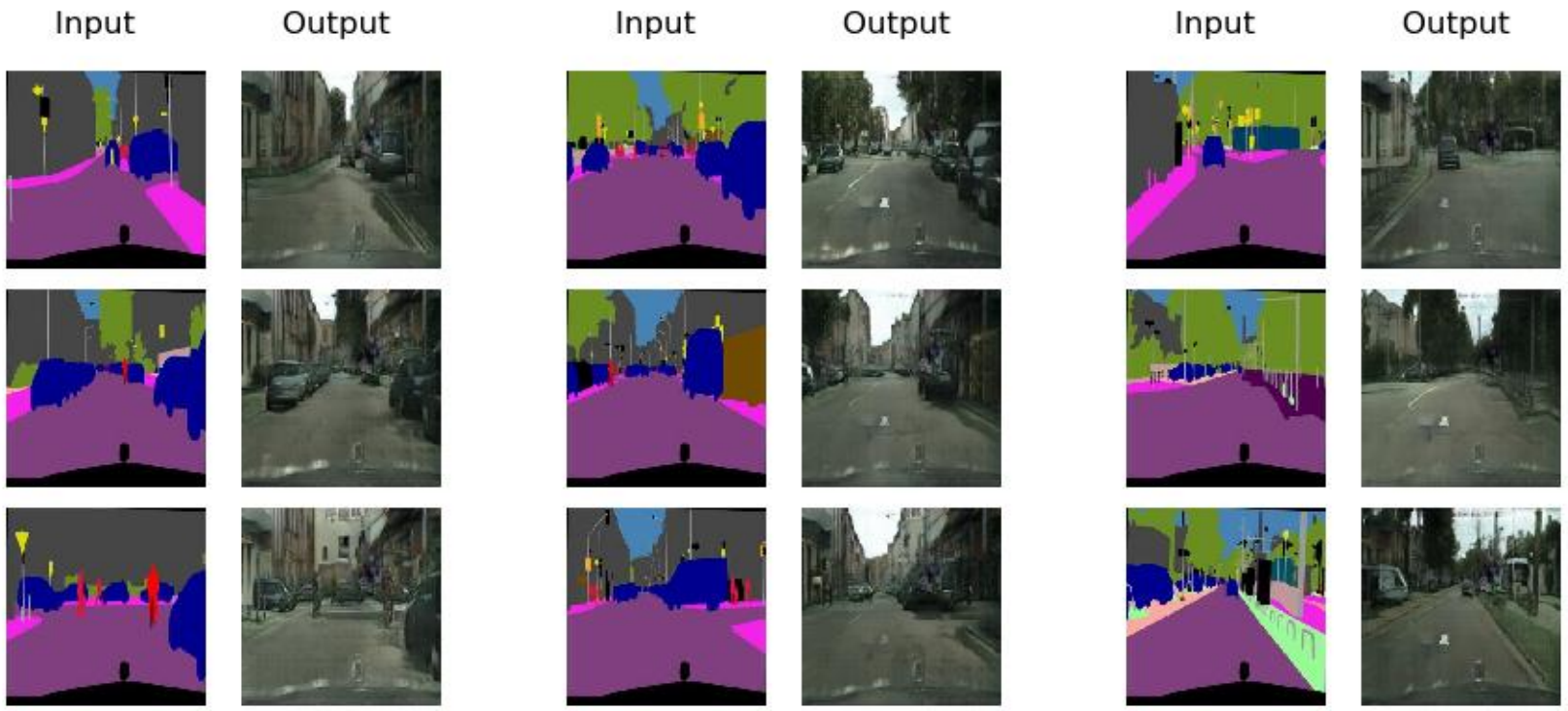
Facades BtoA (architectural labels → photo)



BW to Color



Cityscapes_BtoA



Cityscapes_AtoB